DOCKER

CHEAT SHEETS

A Complete Reference for Docker, Compose & Swarm

Containers · Images · Dockerfile · BuildKit · Volumes · Networking · Compose · Docker Desktop · Swarm · Security · Troubleshooting

31 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & Architecture
02 Installation & CLI Basics
03 Contexts & Configuration

DOCKER DESKTOP

04 Docker Desktop — Install & Settings
05 Docker Desktop — Dashboard
06 Docker Desktop — Desktop-Only Features

RUNNING CONTAINERS

07 Running Containers
08 Container Lifecycle
09 Logs & Inspecting
10 Exec, Copy & Debugging
11 Resource Limits & Runtime Options

IMAGES & REGISTRIES

12 Working with Images
13 Tagging & Registries

DOCKERFILE & BUILDING

14 Dockerfile Basics
15 Dockerfile Configuration Instructions
16 docker build & BuildKit
17 Multi-Stage Builds & Layer Caching
18 Buildx & Multi-Platform Builds

STORAGE & NETWORKING

19 Volumes & Bind Mounts
20 Networking

DOCKER COMPOSE

21 Compose Basics
22 Compose Service Configuration
23 Compose Volumes, Networks & Secrets
24 Compose Workflows

DOCKER SWARM

25 Swarm Setup & Nodes
26 Swarm Services & Rolling Updates
27 Stacks, Secrets & Overlay Networks

OPERATIONS & SECURITY

28 Cleanup & Troubleshooting
29 Security & Best Practices

QUICK REFERENCE

30 Quick Reference — CLI Commands
31 Quick Reference — Dockerfile & Compose

DOCKER

Table of Contents

31 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 03

- 01 Introduction & Archite…
- 02 Installation & CLI Basics
- 03 Contexts & Configurati…

DOCKER DESKTOP 04 – 06

- 04 Docker Desktop — Inst…
- 05 Docker Desktop — Das…
- 06 Docker Desktop — Des…

RUNNING CONTAINERS 07 – 11

- 07 Running Containers
- 08 Container Lifecycle
- 09 Logs & Inspecting
- 10 Exec, Copy & Debuggi…
- 11 Resource Limits & Run…

IMAGES & REGISTRIES 12 – 13

- 12 Working with Images
- 13 Tagging & Registries

DOCKERFILE & BUILDING 14 – 18

- 14 Dockerfile Basics
- 15 Dockerfile Configuratio…
- 16 docker build & BuildKit
- 17 Multi-Stage Builds & L…
- 18 Buildx & Multi-Platfor…

STORAGE & NETWORKING 19 – 20

- 19 Volumes & Bind Mounts
- 20 Networking

DOCKER COMPOSE 21 – 24

- 21 Compose Basics
- 22 Compose Service Con…
- 23 Compose Volumes, Ne…
- 24 Compose Workflows

DOCKER SWARM 25 – 27

- 25 Swarm Setup & Nodes
- 26 Swarm Services & Rolli…
- 27 Stacks, Secrets & Over…

OPERATIONS & SECURITY 28 – 29

- 28 Cleanup & Troublesho…
- 29 Security & Best Practi…

QUICK REFERENCE 30 – 31

- 30 Quick Reference — CLI…
- 31 Quick Reference — Do…

Introduction & Architecture

WHAT DOCKER IS

Docker packages an app with everything it needs (runtime, libraries, config) into an **image**, then runs that image as an isolated **container**. Containers share the host's Linux kernel, so they start in milliseconds and use far less memory than virtual machines. The same image runs the same way on a laptop, a CI runner and a production server.

KEY TERMS

- **Image** — read-only template built from layers (the class)
- **Container** — a running (or stopped) instance of an image (the object)
- **Registry** — server that stores images, e.g. Docker Hub, `ghcr.io`
- **Dockerfile** — recipe that builds an image, one layer per step

HELLO, DOCKER

```
docker run hello-world                # pull a test image, run it, print a message
docker run --rm -it alpine sh         # throwaway shell in a 5 MB Alpine container
docker run -d -p 8080:80 nginx        # web server at http://localhost:8080

docker ps                             # running containers
docker ps -a                          # include stopped ones
docker images                         # images stored locally
docker stop <container>               # SIGTERM, then SIGKILL after 10 s
docker rm <container>                 # delete a stopped container

docker version                        # client + server (engine) versions
docker info                           # storage driver, cgroups, # of containers
```

WHAT HAPPENS ON DOCKER RUN

→ `docker` CLI sends a REST call to the daemon over `/var/run/docker.sock`

→ `dockerd` looks for the image locally, pulls it from the registry if missing

→ `dockerd` hands the container to `containerd`, which manages its lifecycle

→ `runc` creates namespaces + cgroups and starts the process

→ A thin **writable layer** is added on top of the read-only image layers

CONTAINERS VS VIRTUAL MACHINES

ASPECT	CONTAINER	VIRTUAL MACHINE
Isolation	Namespaces + cgroups	Hypervisor, full guest OS
Kernel	Shares the host kernel	Own kernel per VM
Startup	Milliseconds	Seconds to minutes
Size	MBs (app + libs)	GBs (whole OS)
Density	Hundreds per host	Tens per host
OS mix	Linux images need a Linux kernel	Any guest OS

ARCHITECTURE PIECES

COMPONENT	ROLE
docker (CLI)	Client; talks to the daemon's API
dockerd	Daemon: images, networks, volumes, API
containerd	Container lifecycle + image store
runc	Low-level OCI runtime that spawns processes
BuildKit	Build engine behind `docker build`
OCI	Open specs for image format + runtime

⚠ COMMON MISTAKES

- Treating containers like VMs — run one main process per container, not SSH + cron + app
- Expecting data to survive `docker rm` — the writable layer is deleted; use volumes (Sheet 19)
- Running a Windows or macOS app in a Linux image — containers share the host kernel type
- Confusing image and container — `docker rmi` removes images, `docker rm` removes containers

✓ TIPS & BEST PRACTICES

- Add `--rm` to one-off `docker run` commands so stopped containers don't pile up
- Use small official images (`alpine`, `-slim`) to keep pulls fast
- Run `docker info` first when something is off — it shows the storage driver, context and errors
- Docker Desktop runs Linux containers inside a small VM on macOS and Windows

QUICK SUMMARY

- `image` — read-only, layered template
- `container` — image + writable layer + process
- `dockerd → containerd → runc` — the run path
- `registry` — where images are pushed and pulled

Installation & CLI Basics

Desktop vs Engine · docker-ce on Linux · docker version / info · docker group · rootless mode · management commands vs aliases · --format

GETTING DOCKER

On **macOS and Windows**, install **Docker Desktop**: it bundles the engine (inside a lightweight VM), CLI, Compose, Buildx and a GUI (Sheets 04–06). On **Linux servers**, install **Docker Engine** (`docker-ce`) from Docker's own apt/dnf repository; distro packages such as `docker.io` are often older. Docker Desktop for Linux also exists if you want the GUI.

DESKTOP VS ENGINE

- **Engine** — free, open source (Apache 2.0), Linux only
- **Desktop** — free for personal use, education and small businesses (<250 staff and <$10M revenue)
- Larger companies need a paid Desktop subscription
- Both use the same `docker` CLI and commands

INSTALL ENGINE ON UBUNTU/DEBIAN

```
# Quick dev install (convenience script; review it before running)
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Run docker without sudo (log out and back in afterwards)
sudo usermod -aG docker $USER

# Verify
docker version                          # client and server sections
docker run --rm hello-world             # end-to-end test
docker info --format '{{.ServerVersion}} {{.Driver}}'   # 29.x overlayfs

# Rootless mode: daemon runs as your user, not root
dockerd-rootless-setuptool.sh install
```

NOTES

- → `get.docker.com` adds Docker's apt repo and installs `docker-ce`, CLI, containerd, Buildx and Compose plugins
- → For production, follow the manual repo steps in the Docker docs instead of the script
- → The `docker` group is **root-equivalent**: members can mount `/` into a container
- → Rootless mode limits that risk; ports below 1024 need extra setup
- → `--format` takes a Go template, e.g. `'{{json .}}'`

MANAGEMENT COMMANDS VS SHORT ALIASES

MANAGEMENT FORM	SHORT ALIAS
`docker container ls`	docker ps
`docker container run`	docker run
`docker container rm`	docker rm
`docker image ls`	docker images
`docker image rm`	docker rmi
`docker image pull`	docker pull
`docker image build`	docker build

HELP & OUTPUT

COMMAND	DOES
`docker --help`	List all commands
`docker run --help`	Every flag for one command
`docker ps --format table`	Pick columns via Go template
`docker ps -q`	IDs only (for scripting)
`docker info`	Daemon config + status
`docker system info`	Same as `docker info`
`docker <cmd> --format json`	JSON output (most ls/inspect)

⚠ COMMON MISTAKES

- Installing the distro's `docker.io` package and then Docker's `docker-ce` — conflicting packages; remove one first
- Forgetting to log out after `usermod -aG docker` — group changes need a new session (or `newgrp docker`)
- Running `curl | sudo sh` in production — pin versions with the documented repo install instead
- Mixing up client and server versions — `docker version` shows both; the server is what matters

✓ TIPS & BEST PRACTICES

- Prefer the management form (`docker container …`) in scripts; it reads clearly
- Use `docker ps --format 'table {{.Names}}\t{{.Status}}'` for custom columns
- Set `DOCKER_CLI_HINTS=false` to silence CLI tips in scripts
- Keep Engine updated: v29+ rejects clients older than API 1.44 (Docker 25)

QUICK SUMMARY

- `docker-ce` — Engine on Linux
- `Docker Desktop` — Mac/Windows, GUI + VM
- `docker group` — sudo-free, but root-equivalent
- `--format` — Go templates for output

Contexts & Configuration

WHERE THE CLI POINTS

The `docker` CLI is only a client. A **context** stores *which daemon* it talks to (local socket, remote host over SSH, Desktop's VM) plus TLS settings, so you can switch targets with one command. Client settings live in `~/.docker/config.json`; daemon settings live in `/etc/docker/daemon.json` on Linux or in **Settings › Docker Engine** in Docker Desktop.

PRECEDENCE

- `--context` flag beats everything
- then `DOCKER_HOST` / `DOCKER_CONTEXT` env vars
- then the current context from `docker context use`
- `default` context = local socket `unix:///var/run/docker.sock`

CONTEXTS

```
docker context ls                          # * marks the active context
docker context create prod \
  --docker "host=ssh://deploy@prod.example.com"
docker context use prod                    # all commands now hit prod
docker --context default ps                # one-off override
docker context use default                 # back to local

export DOCKER_HOST=ssh://deploy@prod.example.com   # env-var alternative
DOCKER_CONTEXT=prod docker ps                      # per-command env

docker context inspect prod                # endpoint + TLS details
docker context rm prod
```

NOTES

- → SSH contexts need key-based login and Docker on the remote host
- → Desktop creates a `desktop-linux` context; Engine on Linux uses `default`
- → Contexts don't copy files: `docker build .` sends the **local** folder to the remote daemon
- → `docker context export/import` moves a context between machines

~/.DOCKER/CONFIG.JSON (CLIENT)

```
{
  "credsStore": "osxkeychain",
  "currentContext": "default",
  "auths": { "ghcr.io": {} },
  "psFormat": "table {{.Names}}\t{{.Status}}",
  "detachKeys": "ctrl-x,x"
}
```

/ETC/DOCKER/DAEMON.JSON (LINUX DAEMON)

```
{
  "data-root": "/mnt/docker",
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "dns": ["1.1.1.1"],
  "registry-mirrors": ["https://mirror.example.com"],
  "features": { "containerd-snapshotter": true }
}
```

⚠ COMMON MISTAKES

- Editing `daemon.json` with a syntax error — the daemon won't start; validate with `dockerd --validate`
- Changing `daemon.json` without `sudo systemctl restart docker` — settings load only at startup
- Leaving `DOCKER_HOST` exported — every later command silently goes to the remote host
- Storing registry passwords as base64 in `auths` — set `credsStore` to use the OS keychain

✓ TIPS & BEST PRACTICES

- Name contexts after environments (`dev` , `staging` , `prod`) and check `docker context ls` before risky commands
- Use `credsStore` (`osxkeychain` , `wincred` , `pass` , `secretservice`) to keep tokens out of plain text
- Set default log rotation in `daemon.json` so `json-file` logs can't fill the disk
- `DOCKER_CONFIG=/path` points the CLI at a different config folder

QUICK SUMMARY

- `docker context use` — switch target daemon
- `DOCKER_HOST` — env override, e.g. `ssh://…`
- `config.json` — client prefs + credentials
- `daemon.json` — engine settings (restart needed)

Sheet 04

Docker Desktop — Install & Settings

macOS / Windows / Linux installs · WSL 2 backend · CPU / memory / disk limits · file sharing (VirtioFS) · Rosetta on Apple silicon · docker desktop start / stop / status · license terms

WHAT DOCKER DESKTOP ADDS

Docker Desktop runs the Linux engine inside a managed VM and adds the CLI, Compose, Buildx, a dashboard, Kubernetes and extensions in one installer. Containers can only use the CPU, memory and disk you give that VM, so the **Resources** settings matter. On Windows the VM is a **WSL 2** distro; on macOS it uses Apple's Virtualization framework.

INSTALL

- **macOS** — `.dmg` from docker.com or `brew install --cask docker`
- **Windows** — installer; enable WSL 2 first (`wsl --install`)
- **Linux** — `.deb` / `.rpm` package; runs its own VM, separate from Engine
- Free for personal, education and small business; paid for larger orgs

DOCKER DESKTOP CLI

```
docker desktop status             # running / stopped
docker desktop start              # start the VM + engine (no GUI needed)
docker desktop stop               # stop everything
docker desktop restart
docker desktop update --check-only
docker desktop logs               # Desktop's own log entries
docker desktop diagnose           # gather + upload diagnostics
docker desktop enable model-runner      # turn a feature on/off
docker desktop engine use windows       # Windows only: switch engine
```

NOTES

- Handy for scripts and CI agents: start Desktop headless, wait for `status`
- `engine ls/use` switch between Linux and Windows containers (Windows hosts only)
- `docker desktop kubernetes` manages the built-in cluster (Sheet 06)
- Settings changes in the GUI may restart the VM and stop running containers

KEY SETTINGS

SETTINGS PATH	CONTROLS
General › Start at login	Auto-start Desktop
General › Use containerd for images	Image store (needed for multi-platform)
Resources › Advanced	CPUs, memory, swap, disk limit
Resources › File sharing	Host folders allowed in bind mounts
Resources › Proxies	HTTP/HTTPS proxy for pulls + containers
Docker Engine	Edit `daemon.json` in the GUI
Kubernetes	Enable the built-in cluster

PLATFORM NOTES

PLATFORM	WATCH OUT FOR
Apple silicon	Images are arm64; Rosetta runs amd64 faster
Apple silicon	Add `--platform linux/amd64` for x86-only images
macOS	VirtioFS file sharing is fastest for bind mounts
Windows	Keep code inside WSL (`\\wsl$`) for speed
Windows	WSL memory set in `.wslconfig`, not Desktop
All	Disk image grows; `docker system prune` frees it

⚠ COMMON MISTAKES

- Mounting a host folder that isn't shared — add it under **Resources › File sharing** (macOS/Linux)
- Running a bind-mounted Node/PHP project from the Windows filesystem — very slow; move it into the WSL distro
- Giving the VM 2 GB and wondering why builds get `OOMKilled` — raise memory under Resources
- Installing Desktop for Linux next to Engine and confusing contexts — check `docker context ls`

✓ TIPS & BEST PRACTICES

- Turn on **Rosetta for x86/amd64 emulation** on Apple silicon for faster amd64 containers
- Use `docker desktop start` in login scripts instead of opening the GUI
- Enable the containerd image store to build and load multi-platform images locally
- Set a sane disk limit; Desktop's VM disk does not shrink on its own

QUICK SUMMARY

- `Docker Desktop` — VM + engine + GUI + tools
- `WSL 2` — the Windows backend
- `Resources` — CPU/memory/disk for all containers
- `docker desktop start|stop|status` — CLI control

Docker Desktop — Dashboard

THE DASHBOARD

The Docker Desktop window is a GUI over the same engine the CLI uses: anything you start with `docker run` or `docker compose up` shows up here, and actions taken here are ordinary API calls. It is most useful for browsing logs, poking around inside a container's files, cleaning up space and reading Scout vulnerability reports.

GOOD TO KNOW

- Compose projects appear as a group you can start/stop together
- Search bar finds images on Docker Hub and local objects
- The built-in **Terminal** opens a host shell inside Desktop
- Some views need a Docker account sign-in

VIEWS

VIEW	WHAT YOU CAN DO
`Containers`	Start/stop/delete, open ports in browser, group by Compose project
`Container › Logs`	Live, searchable logs (same as `docker logs -f`)
`Container › Inspect`	Env vars, mounts, ports (same as `docker inspect`)
`Container › Exec`	Shell inside the container; Debug mode for slim images
`Container › Files`	Browse, edit, import/export files in the container
`Container › Stats`	CPU, memory, disk and network graphs
`Images`	Local + Hub images, sizes, layers, pull/push, run with options
`Volumes`	Size, contents, clone, export to file or registry
`Builds`	Build history, timing per step, logs, cache use, errors
`Models`	Pull and run local AI models (Model Runner)

DOCKER SCOUT IN THE GUI

WHERE	SHOWS
`Images › image › Vulnerabilities`	CVEs by severity per layer
`Images › image › Packages`	SBOM: every package + version
`Recommended fixes`	Newer base images with fewer CVEs
`CLI equivalent`	`docker scout quickview`, `cves`

EXTENSIONS & GORDON

FEATURE	USE
`Extensions › Browse`	Install add-ons (disk usage, logs, DBs)
`Extensions › Manage`	Update or remove installed ones
`Ask Gordon`	Built-in AI assistant: explain errors, write Dockerfiles
`docker ai "…"`	Ask Gordon from the CLI

⚠ COMMON MISTAKES

- Thinking the GUI shows a different engine than the CLI — check the CLI's active context
- Deleting a volume from the Volumes view — the data is gone for good; export it first
- Editing files through **Files** and expecting them to persist — they're lost when the container is removed
- Installing many extensions — each runs containers and uses VM resources

✓ TIPS & BEST PRACTICES

- Use **Builds** to find the slow step in a Dockerfile; it shows per-step timing and cache hits
- Use **Exec › Debug mode** to get tools like `curl` and `vim` inside slim or distroless containers
- Group by Compose project to restart a whole stack with one click
- Treat Gordon's answers as drafts; review generated Dockerfiles before using them

QUICK SUMMARY

- `Containers` — logs, exec, files, stats
- `Images` — layers, Scout CVEs, SBOM
- `Builds` — history and step timing
- `Extensions` / `Ask Gordon` — add-ons and AI help

Sheet 06

Docker Desktop — Desktop-Only Features

built-in Kubernetes · host.docker.internal · Model Runner (docker model) · Enhanced Container Isolation · proxy settings · Troubleshoot / reset to factory defaults

FEATURES ENGINE DOESN'T HAVE

Docker Desktop adds a few things plain Docker Engine on Linux lacks: a one-click local **Kubernetes** cluster, the special DNS name **host.docker.internal** for reaching your host, **Model Runner** for local AI models, admin-level security controls, and a troubleshooting menu that can reset the VM without reinstalling.

NOTES

- On Linux Engine, add `--add-host=host.docker.internal:host-gateway` to get the same name
- **Enhanced Container Isolation** and Settings Management need a Docker Business plan
- Kubernetes here is for local dev; see the Kubernetes set for `kubectl`

USING THE EXTRAS

```
# Reach a service running on your laptop (e.g. Postgres on :5432)
docker run --rm postgres:17 psql -h host.docker.internal -U app

# Built-in Kubernetes (after enabling it in Settings › Kubernetes)
kubectl config use-context docker-desktop
kubectl get nodes

# Model Runner: local LLMs with an OpenAI-compatible API
docker model pull ai/smollm2
docker model run ai/smollm2 "Explain Docker volumes in one line"
docker model list                # models stored locally
docker model ps                  # models loaded in memory
docker model rm ai/smollm2
```

NOTES

- → `host.docker.internal` resolves to the host from inside any container
- → `gateway.docker.internal` reaches the VM's gateway
- → Desktop Kubernetes can use `kubeadm` (single node) or `kind` (multi-node)
- → Model Runner models are OCI artifacts pulled from Docker Hub or Hugging Face
- → Compose can declare models with a top-level `models:` key

TROUBLESHOOT MENU (BUG ICON)

ACTION	EFFECT
`Restart Docker Desktop`	Restarts VM + engine
`Clean / Purge data`	Deletes images, containers, volumes
`Reset Kubernetes cluster`	Wipes the local cluster
`Reset to factory defaults`	Settings + all data back to new
`Gather diagnostics`	Same as `docker desktop diagnose`

ADMIN & SECURITY

FEATURE	DOES
`Enhanced Container Isolation`	Runs containers in user namespace; blocks privileged escapes
`Settings Management`	Org-wide locked settings (`admin-settings.json`)
`Registry Access Mgmt`	Allow-list registries users can pull from
`Proxies`	Manual or system proxy for VM + containers

⚠ COMMON MISTAKES

- Using `localhost` inside a container to reach the host — that's the container itself; use `host.docker.internal`
- Hard-coding `host.docker.internal` in prod config — it exists only on Desktop (or with `host-gateway`)
- Clicking **Clean / Purge data** without backups — every volume is deleted
- Leaving Kubernetes enabled when unused — it uses CPU and memory all the time

✓ TIPS & BEST PRACTICES

- Try **Restart** first, then **Reset Kubernetes**, before **Reset to factory defaults**
- Use `extra_hosts: ["host.docker.internal:host-gateway"]` in Compose so it works on Linux too
- Run `docker model status` to check Model Runner before calling its API
- Corporate proxies: set them in **Resources › Proxies** so both pulls and containers use them

QUICK SUMMARY

- `host.docker.internal` — reach the host
- `docker-desktop` — local Kubernetes context
- `docker model` — run local AI models
- `Troubleshoot` — restart, reset, diagnose

Running Containers

DOCKER RUN

`docker run` = **create + start** in one step: it pulls the image if needed, creates a container with your options, then starts its main process. Everything after the image name replaces the image's default command (`CMD`). Options always go **before** the image name; arguments for the app go after it.

SHAPE OF THE COMMAND

- `docker run [OPTIONS] IMAGE[:TAG] [COMMAND] [ARG...]`
- No tag = `:latest`, which is just a tag name, not "newest"
- `docker create` + `docker start` split the two steps
- Container exits when its main process (PID 1) exits

COMMON PATTERNS

```
docker run -d --name web -p 8080:80 nginx:1.27        # background service
docker run --rm -it ubuntu:24.04 bash                 # interactive, auto-delete
docker run --rm alpine echo "hi"                      # override CMD: prints hi
docker run --rm -e TZ=UTC -e DEBUG=1 alpine env       # set env vars
docker run --rm --env-file .env myapp                 # env vars from a file
docker run --rm -w /src -v "$PWD":/src node:22 npm test  # workdir + mount
docker run --rm --entrypoint sh myapp -c 'ls /app'    # new ENTRYPOINT
docker run -d --name db -e POSTGRES_PASSWORD=dev \
  -v pgdata:/var/lib/postgresql/data postgres:17      # named volume
docker run --rm --network host nginx                  # host network (Linux)
```

CODE EXPLAINED

- `-d` — detached: runs in background, prints the container ID
- `-it` — `-i` keeps stdin open + `-t` allocates a terminal; use both for shells
- `--rm` — delete the container when it exits
- `--name` — fixed name instead of a random one like `jolly_turing`
- `-w` — working directory inside the container
- `--entrypoint` — replaces the image's ENTRYPOINT; args after the image go to it

MORE DOCKER RUN FLAGS

FLAG	DOES
`-p 8080:80`	Publish container port 80 on host 8080
`-P`	Publish all EXPOSEd ports on random host ports
`-v name:/path`	Mount a volume (Sheet 19)
`--restart unless-stopped`	Restart policy (Sheet 08)
`-u 1000:1000`	Run as user/group ID
`--label env=dev`	Metadata for filtering
`--pull always`	Always check for a newer image
`--platform linux/amd64`	Pick image architecture

.ENV FILE FORMAT (--ENV-FILE)

LINE	RESULT
`DB_HOST=db`	Sets DB_HOST to db
`MSG=hello world`	Spaces are kept; no quotes needed
`QUOTED="x"`	Quotes are kept literally
`# comment`	Ignored
`HOME_DIR`	Copies the value from your shell
`-e VAR (no =)`	Same: pass through from shell

⚠ COMMON MISTAKES

- Putting options after the image — `docker run nginx -p 80:80` passes `-p` to nginx, not Docker
- Using `-it` in scripts or CI — no TTY there; drop `-t` or the command fails
- Expecting `docker run` to reuse a container — it always makes a new one; use `docker start` to restart
- Name conflicts — `--name web` fails if a stopped `web` exists; `docker rm web` or add `--rm`

✓ TIPS & BEST PRACTICES

- Pin tags (`nginx:1.27`) instead of `latest` for repeatable results
- Use `--env-file` for many variables and keep secrets out of shell history
- Detach from `-it` without stopping: press `Ctrl-P Ctrl-Q`
- `docker run --init` adds a tiny init so signals and zombies are handled (Sheet 11)

QUICK SUMMARY

- `-d` / `-it` — background vs interactive
- `--rm` — auto-delete on exit
- `-e` / `--env-file` — environment variables
- `IMAGE CMD args` — args after the image replace CMD

Container Lifecycle

STATES

A container moves through **created → running → (paused) → exited**, and finally **removed**. Stopping keeps the container and its writable layer, so you can start it again with the same config; removing deletes it. Restart policies tell the daemon to bring a container back after it exits or after the host reboots.

STOP VS KILL

- `stop` sends **SIGTERM**, waits 10 s, then **SIGKILL**
- `kill` sends SIGKILL right away (or `-s` another signal)
- Image `STOPSIGNAL` changes the first signal
- `stop -t 30` waits 30 s before forcing

LIFECYCLE COMMANDS

```
docker ps                          # running only
docker ps -a                       # all, including exited
docker ps -a --filter status=exited --format '{{.Names}}: {{.Status}}'
docker start web                   # start an existing (stopped) container
docker start -ai web               # start and attach (interactive)
docker stop web                    # graceful stop
docker stop -t 30 web              # longer grace period
docker restart web
docker kill web                    # immediate SIGKILL
docker kill -s HUP web             # send SIGHUP (e.g. reload config)
docker pause web                   # freeze all processes (unpause to thaw)
docker rm web                      # remove stopped container
docker rm -f web                   # stop + remove in one step
docker container prune             # remove ALL stopped containers
docker wait web                    # block until it exits, print exit code
```

NOTES

- → Most commands take a name, a full ID or a unique ID prefix (`docker stop 3f2`)
- → Pass several at once: `docker stop web db cache`
- → `docker rm $(docker ps -aq -f status=exited)` — remove exited ones
- → `docker update --restart=no web` changes a policy without recreating

--RESTART POLICIES

POLICY	RESTARTS WHEN…	AFTER DAEMON RESTART
`no`	Never (default)	Stays stopped
`on-failure[:5]`	Exit code ≠ 0 (max 5 tries)	Only if it had been running
`always`	Any exit	Always started
`unless-stopped`	Any exit	Started unless you stopped it

DOCKER PS STATUS COLUMN

STATUS	MEANING
`Up 3 minutes`	Running
`Up … (healthy)`	Running + passing HEALTHCHECK
`Exited (0)`	Finished OK
`Exited (137)`	Killed (SIGKILL / out of memory)
`Restarting (1)`	Crash loop under a restart policy
`Created`	Never started

⚠ COMMON MISTAKES

- Expecting `docker stop` to be instant — apps that ignore SIGTERM wait the full 10 s, then get killed
- Using `always` for one-off jobs — they restart forever; use `on-failure` or `no`
- `docker container prune` on a shared host — deletes everyone's stopped containers
- Debugging a crash loop by restarting — read `docker logs` first (Sheet 09)

✓ TIPS & BEST PRACTICES

- Use `unless-stopped` for long-running services on a single host
- Make your app handle SIGTERM to shut down cleanly within the grace period
- Filter with `-f name=`, `-f label=`, `-f status=`, `-f ancestor=nginx`
- Use `docker wait` in scripts to get a job's exit code

QUICK SUMMARY

- `stop` / `kill` — SIGTERM+grace vs SIGKILL
- `rm -f` — force stop + remove
- `--restart unless-stopped` — survive reboots
- `ps -a` — see exited containers too

Logs & Inspecting

SEEING INSIDE

Docker captures everything a container's main process writes to **stdout and stderr**; `docker logs` reads it back. `docker inspect` returns the full JSON config and state of any object (container, image, volume, network), and `--format` pulls out single fields with Go templates. `stats`, `top` and `events` show live resource use, processes and daemon activity.

NOTES

- Apps must log to stdout/stderr, not files, for `docker logs` to work
- Logs persist until the container is removed
- `docker logs` works with `json-file` and `local` drivers (and others via dual logging)

LOGS AND INSPECT

```
docker logs web                          # everything so far
docker logs -f web                       # follow (like tail -f)
docker logs --tail 50 -f web             # last 50 lines, then follow
docker logs --since 10m web              # last 10 minutes (or 2026-10-16T09:00)
docker logs -t web 2>&1 | grep ERROR     # timestamps; merge stderr to grep

docker inspect web                       # full JSON
docker inspect -f '{{.State.Status}} {{.State.ExitCode}}' web
docker inspect -f '{{.NetworkSettings.Networks.bridge.IPAddress}}' web
docker inspect -f '{{json .Mounts}}' web | jq
docker inspect -f '{{.State.Health.Status}}' web     # healthy / unhealthy
docker inspect -f '{{.Config.Env}}' web              # env vars
```

TEMPLATE TIPS

- → `{{.Field.Sub}}` — walk the JSON keys shown by plain `inspect`
- → `{{json .X}}` — print a sub-object as JSON
- → `{{range .Mounts}}{{.Source}} {{end}}` — loop over a list
- → `{{index .Config.Labels "env"}}` — keys with dots or dashes
- → Pipe to `jq` for anything complicated

LIVE VIEWS

COMMAND	SHOWS
`docker stats`	Live CPU %, memory, net + disk I/O
`docker stats --no-stream`	One snapshot (for scripts)
`docker top web`	Processes inside (like `ps`)
`docker port web`	Port mappings, e.g. `80/tcp -> 0.0.0.0:8080`
`docker diff web`	Files changed: A added, C changed, D deleted
`docker events`	Live stream: start, die, oom, pull…
`docker events -f type=container`	Filter by object/event type

USEFUL INSPECT FIELDS

TEMPLATE	VALUE
`.State.Status`	running, exited, …
`.State.OOMKilled`	true if killed for memory
`.State.StartedAt`	Start timestamp
`.RestartCount`	Times restarted by policy
`.Config.Image`	Image the container uses
`.HostConfig.RestartPolicy`	Restart policy
`.LogPath`	Host path of the log file

⚠ COMMON MISTAKES

- Logging to `/var/log/app.log` inside the container — `docker logs` shows nothing; symlink it to `/dev/stdout`
- Grepping `docker logs` without `2>&1` — stderr lines skip the pipe
- No log rotation — `json-file` grows forever; set `max-size` (Sheets 03, 28)
- Reading the IP from `inspect` to connect containers — use network names instead (Sheet 20)

✓ TIPS & BEST PRACTICES

- `docker logs --since 5m -f` is the fastest way to watch a misbehaving service
- `docker events --filter event=oom` catches memory kills as they happen
- Save a template: `docker ps --format '{{.Names}}\t{{.Image}}'` in `psFormat`
- `docker diff` quickly shows what an app wrote to the container filesystem

QUICK SUMMARY

- `logs -f --tail N --since T` — read output
- `inspect -f '{{…}}'` — any config/state field
- `stats` / `top` — resources and processes
- `events` / `diff` / `port` — activity, file changes, ports

Sheet 10

Exec, Copy & Debugging

docker exec -it sh · exec -u root · attach vs exec · docker cp · docker debug · commit · exit codes 125 / 126 / 127

GETTING INTO A CONTAINER

`docker exec` starts a **new process** inside a running container — usually a shell for poking around. `docker attach` instead connects your terminal to the **main process** (PID 1), so Ctrl-C there can stop the container. `docker cp` copies files in or out, even from stopped containers, and `docker debug` gives you a full toolbox shell in slim images that have no shell at all.

EXEC VS ATTACH

- `exec` — new process; exiting it leaves the container running
- `attach` — main process's stdin/stdout; detach with `Ctrl-P Ctrl-Q`
- `exec` only works on **running** containers
- Use `docker logs -f` rather than `attach` to watch output

EXEC, COPY, DEBUG

```
docker exec -it web sh                  # shell (bash may not exist)
docker exec -it web bash
docker exec -it -u root web sh          # as root, even if image uses USER
docker exec web cat /etc/os-release     # one command, no TTY
docker exec -e DEBUG=1 -w /app web ./check.sh
docker exec -it db psql -U postgres     # open a DB client inside

docker cp web:/etc/nginx/nginx.conf .   # container → host
docker cp ./site/. web:/usr/share/nginx/html/   # host → container
docker cp web:/var/log - | tar -t       # stream a tar archive

docker debug web                        # toolbox shell (Desktop)
docker debug myimage:1.0                # debug an image without running it
docker commit web web-snapshot:debug    # save container state as an image
```

NOTES

→ Distroless/scratch images have no `sh`, so `exec` fails; use `docker debug`

→ `docker debug` changes to a running container are real; for images they're discarded

→ `docker cp` paths: trailing `/.` copies a folder's **contents**

→ `docker commit` is for debugging snapshots, not for building images

EXIT CODES

CODE	MEANING	USUAL FIX
0	Success	—
1	App error	Read `docker logs`
125	`docker run` itself failed	Bad flag, name taken, daemon error
126	Command found but not executable	`chmod +x`, wrong format
127	Command not found	Typo or missing in image (`bash` on Alpine)
137	SIGKILL (128+9)	OOM or `docker kill`/stop timeout
139	Segfault (128+11)	App crash, wrong architecture
143	SIGTERM (128+15)	Normal `docker stop`

⚠ COMMON MISTAKES

- `docker exec -it app bash` on Alpine — `bash` isn't installed; use `sh`
- Fixing a bug inside a running container with `exec` — lost on the next deploy; change the Dockerfile
- Pressing Ctrl-C while attached — stops the main process and often the container
- `docker cp ./dir web:/app` vs `./dir/.` — the first nests `dir` inside `/app`

✓ TIPS & BEST PRACTICES

- Check `docker inspect -f '{{.State.ExitCode}}'` before digging through logs
- Use `docker exec web env` to confirm env vars actually reached the app
- Crash on start? Override the command: `docker run -it --entrypoint sh myimage`
- Exit 139 on Apple silicon often means an amd64-only image; add `--platform`

QUICK SUMMARY

- `exec -it … sh` — shell in a running container
- `cp src dest` — files in/out (`name:/path`)
- `debug` — toolbox for slim images
- `125/126/127/137` — docker, not-exec, not-found, killed

Resource Limits & Runtime Options

LIMITS AND HARDENING

By default a container can use **all** of the host's CPU and memory. Limits use Linux **cgroups**: hit the memory limit and the kernel kills the process (**OOMKilled**, exit 137); hit the CPU limit and it's only slowed down. Runtime flags such as `--user` and `--read-only` also shrink what a compromised app could do.

UNITS

- Memory: `b`, `k`, `m`, `g` — e.g. `512m`, `2g`
- `--cpus 1.5` = one and a half cores' worth of time
- On Desktop, limits sit inside the VM's own limits
- Change limits live: `docker update --memory 1g web`

SETTING LIMITS

```
docker run -d --name api \
  --memory 512m --memory-swap 512m \
  --cpus 1.5 --pids-limit 200 \
  --user 1000:1000 \
  --read-only --tmpfs /tmp:size=64m \
  --init --ulimit nofile=65536:65536 \
  myapi:1.4

docker stats --no-stream api              # check usage vs limit
docker inspect -f '{{.State.OOMKilled}}' api    # true → raise --memory
docker update --cpus 2 --memory 1g --memory-swap 1g api
```

CODE EXPLAINED

- `--memory-swap` = memory + swap; equal to `--memory` means no swap
- `--pids-limit` caps processes (stops fork bombs)
- `--cpus` throttles; it never kills the container
- `--read-only` plus `--tmpfs` keeps only `/tmp` writable (in RAM)
- `--init` reaps zombie processes and forwards signals to your app
- `--ulimit` raises per-process limits like open files

CPU & MEMORY FLAGS

FLAG	EFFECT
`--memory / -m`	Hard memory limit
`--memory-reservation`	Soft limit under host pressure
`--oom-score-adj`	Make OOM kill less/more likely
`--cpus`	Max CPU time
`--cpuset-cpus 0,1`	Pin to specific cores
`--cpu-shares 512`	Relative weight (default 1024)
`--gpus all`	Expose NVIDIA GPUs (needs toolkit)

RUNTIME OPTIONS

FLAG	EFFECT
`--user / -u`	Override the image's USER
`--read-only`	Root filesystem read-only
`--tmpfs /run`	In-memory writable folder
`--init`	Small init process as PID 1
`--shm-size 1g`	Bigger `/dev/shm` (browsers, DBs)
`--stop-timeout 30`	Grace period for this container
`--sysctl net.core.somaxconn=1024`	Kernel param (namespaced only)

⚠ COMMON MISTAKES

- Setting `--memory` below what the app (or JVM heap) needs — instant exit 137; check `OOMKilled`
- Expecting `--cpus 0.5` to fail when exceeded — it only throttles, so things just get slow
- `--read-only` without `--tmpfs` for `/tmp` or `/run` — apps that write temp files crash
- Running Chrome/Playwright with the default 64 MB `/dev/shm` — random crashes; raise `--shm-size`

✓ TIPS & BEST PRACTICES

- Set both `--memory` and `--cpus` on shared hosts so one container can't starve the rest
- Java 10+, Node 20+ and Go read cgroup limits; size heaps relative to `--memory`
- Use `--init` (or `init: true` in Compose) for apps that spawn child processes
- Prefer `--user` with a fixed UID that owns the files your app writes

QUICK SUMMARY

- `--memory` — hard cap; exceed = OOMKilled
- `--cpus` — throttle, never kill
- `--read-only` + `--tmpfs` — locked-down filesystem
- `docker update` — change limits live

Working with Images

IMAGES, TAGS AND DIGESTS

An image reference looks like `registry/namespace/name:tag@digest`. A **tag** (`1.27`, `latest`) is a movable label the publisher can point at a new build; a **digest** (`sha256:…`) is the content hash and never changes. Images are stacks of read-only **layers**, and layers shared between images are stored only once on disk.

READING A REFERENCE

- `nginx` = `docker.io/library/nginx:latest`
- `bitnami/redis:7.4` — Docker Hub user/org `bitnami`
- `ghcr.io/acme/api:2.1` — another registry
- `nginx@sha256:ab12…` — exact, immutable content

PULL, LIST, REMOVE

```
docker pull nginx:1.27                       # one tag
docker pull --platform linux/arm64 nginx:1.27
docker pull nginx@sha256:<digest>            # exact content

docker image ls                              # same as docker images
docker image ls --digests                    # show sha256 digests
docker image ls -f dangling=true             # untagged <none> layers

docker image history nginx:1.27              # layers + the step that made each

docker rmi nginx:1.27                        # remove tag (+ layers if unused)
docker image prune                           # dangling only
docker image prune -a                        # every image no container uses
```

NOTES

- → `rmi` fails while a container (even stopped) uses the image; add `-f` or remove it first
- → **Dangling** = `<none>:<none>`, left over when a tag moves to a new build
- → `history --no-trunc` shows full `RUN` commands
- → Size shown is the uncompressed size on disk

MOVING IMAGES WITHOUT A REGISTRY

COMMAND	DOES
`docker save -o app.tar app:1.0`	Image(s) → tar file (layers + tags)
`docker load -i app.tar`	tar → image store
`docker save app:1.0 | gzip > app.tgz`	Compressed archive
`docker save app | ssh host docker load`	Copy to another host
`docker export web > fs.tar`	Container filesystem only (flat)
`docker import fs.tar app:flat`	Flat tar → single-layer image

IMAGE LS FILTERS

FILTER	MATCHES
`dangling=true`	Untagged images
`reference=api:*`	Name/tag pattern
`label=team=web`	Images with a label
`since=nginx:1.27`	Created later

⚠ COMMON MISTAKES

- Relying on `latest` — it can change any time; two hosts may run different code
- `docker image prune -a` on a build server — wipes base images, so the next build re-downloads everything
- Using `export` / `import` to back up an image — it flattens layers and drops CMD, ENV and other config
- Thinking `docker rmi` frees space while containers still use the layers — remove containers first

✓ TIPS & BEST PRACTICES

- Pin by digest in production: `FROM python:3.13-slim@sha256:…`
- Use `docker image ls --digests` to grab the digest of what you just tested
- `docker save` / `load` is the way to ship images to air-gapped machines
- Check `docker image history` to find which step made an image huge

QUICK SUMMARY

- `pull name:tag` — download
- `tag` vs `@sha256:` — movable vs fixed
- `rmi` / `image prune -a` — clean up
- `save` / `load` — images as tar files

Tagging & Registries

SHARING IMAGES

To push, an image needs a name that includes **where** it goes: `registry-host/namespace/repo:tag`. `docker tag` adds that name (it doesn't copy anything), `docker login` stores credentials, and `docker push` uploads only the layers the registry doesn't already have. With no host in the name, Docker assumes Docker Hub (`docker.io`).

NOTES

- One image can carry many tags: `1.4.2`, `1.4`, `1`, `latest`
- Docker Hub free accounts have pull rate limits; log in to raise them
- Use access tokens, not passwords, for `docker login`

TAG, LOG IN, PUSH

```
docker tag myapp:dev myuser/myapp:1.4.2          # Docker Hub
docker tag myapp:dev myuser/myapp:latest
docker login                                     # Docker Hub (browser login)
docker push myuser/myapp:1.4.2
docker push -a myuser/myapp                      # all local tags of the repo

# GitHub Container Registry
echo "$GH_TOKEN" | docker login ghcr.io -u octocat --password-stdin
docker tag myapp:dev ghcr.io/acme/myapp:1.4.2
docker push ghcr.io/acme/myapp:1.4.2

# Local private registry
docker run -d -p 5000:5000 --name registry registry:2
docker tag myapp:dev localhost:5000/myapp:1.4.2
docker push localhost:5000/myapp:1.4.2
docker logout ghcr.io
```

NOTES

- → `--password-stdin` keeps tokens out of shell history and `ps`
- → `push -a` pushes every tag of that repository
- → `localhost:5000` is trusted over HTTP; other hosts need TLS or `insecure-registries` in `daemon.json`
- → Multi-arch pushes use `docker buildx build --push` (Sheet 18)

REGISTRY LOGINS

REGISTRY	NAME FORMAT	LOGIN
Docker Hub	user/repo:tag	`docker login`
GitHub (GHCR)	ghcr.io/org/repo	PAT with `write:packages`
AWS ECR	<acct>.dkr.ecr.<region>.amazonaws.com/repo	`aws ecr get-login-password | docker login …`
Azure ACR	<name>.azurecr.io/repo	`az acr login -n <name>`
Google AR	<region>-docker.pkg.dev/proj/repo/img	`gcloud auth configure-docker`
GitLab	registry.gitlab.com/group/proj	Deploy/job token

⚠ COMMON MISTAKES

- `docker push myapp:1.0` with no namespace — Hub denies it; tag as `user/myapp:1.0`
- Pushing only `latest` — you can't roll back; push a version tag too
- Pushing an arm64-only image from a Mac to amd64 servers — `exec format error`; build multi-platform
- Pasting tokens with `-p` — they land in shell history; use `--password-stdin`

✓ TIPS & BEST PRACTICES

- Tag with the git SHA as well as semver for exact traceability
- Use `docker manifest inspect` or `buildx imagetools inspect` to see which platforms a tag has
- Credential helpers (`docker-credential-ecr-login`, `gcloud`) refresh short-lived tokens for you
- Set a `registry-mirrors` cache in `daemon.json` to dodge Hub rate limits in CI

QUICK SUMMARY

- `tag src dst` — add a name, no copy
- `login` / `push` — authenticate, upload
- `host/ns/repo:tag` — where it goes
- `registry:2` — run your own registry

Dockerfile Basics

FROM · RUN · COPY vs ADD · WORKDIR · EXPOSE · CMD vs ENTRYPOINT · exec vs shell form · .dockerignore

THE RECIPE

A **Dockerfile** is a list of instructions run top to bottom to build an image. `FROM` picks a base image, `RUN`, `COPY` and `ADD` create new **layers**, and `CMD`/`ENTRYPOINT` set what runs when a container starts. Each layer is cached, so the order of instructions decides how fast rebuilds are (Sheet 17).

NOTES

- File is named `Dockerfile` (no extension) by convention
- `#` comments only at the **start** of a line; `\` continues one
- The **build context** is the folder you pass to `docker build`

A COMPLETE NODE.JS DOCKERFILE

```
FROM node:22-alpine
WORKDIR /app
# Dependencies first, so this layer stays cached
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
# Then the source (filtered by .dockerignore)
COPY . .
# Documents the port; you still need -p to publish
EXPOSE 3000
USER node
# Exec form: node runs as PID 1
CMD ["node", "server.js"]
```

.DOCKERIGNORE

```
node_modules
.git
*.log
.env
Dockerfile
dist/
```

CMD VS ENTRYPOINT

DOCKERFILE	DOCKER RUN IMG	DOCKER RUN IMG X
`CMD ["npm","start"]`	npm start	x (CMD replaced)
`ENTRYPOINT ["ping"]`	ping	ping x
`ENTRYPOINT ["ping"] + CMD ["localhost"]`	ping localhost	ping x
`ENTRYPOINT ping (shell form)`	/bin/sh -c ping	args ignored

EXEC VS SHELL FORM

FORM	BEHAVIOR
`CMD ["node","app.js"]`	Exec: no shell, gets signals directly
`CMD node app.js`	Shell: `/bin/sh -c`, sh is PID 1
`RUN apt-get install -y curl`	Shell form fine for RUN

COPY VS ADD

INSTRUCTION	DOES
`COPY src dest`	Copy files from the build context
`COPY --chown=node:node . .`	Copy and set owner
`COPY --from=build /out /app`	Copy from another stage (Sheet 17)
`ADD app.tar.gz /opt/`	Copies **and extracts** local tar files

⚠ COMMON MISTAKES

- Shell-form `CMD node app.js` — `sh` becomes PID 1 and swallows SIGTERM, so stops take 10 s
- No `.dockerignore` — `node_modules`, `.git` and `.env` get sent and baked into the image
- `RUN cd /app` — each RUN is a new shell; use `WORKDIR /app` instead
- `EXPOSE` doesn't publish anything — you still need `-p` on `docker run`

✓ TIPS & BEST PRACTICES

- Use `COPY` unless you need tar extraction; `ADD`'s magic surprises people
- Copy dependency manifests first, install, **then** copy source to keep the install layer cached
- `ENTRYPOINT` for the fixed program, `CMD` for default arguments users can override
- Run `docker init` to generate a starter Dockerfile + `.dockerignore` for your language

QUICK SUMMARY

- `FROM` — base image
- `RUN` / `COPY` — build steps, one layer each
- `CMD` vs `ENTRYPOINT` — default args vs fixed program
- `.dockerignore` — keep junk out of the context

Dockerfile Configuration Instructions

CONFIGURING THE IMAGE

These instructions don't add files; they set **metadata and defaults** baked into the image: environment variables, the user to run as, a health probe, labels and signals. ARG values exist only during the build, while ENV values stay in the image and are visible to every container started from it.

ENV VS ARG

- `ARG` — build-time only; set with `--build-arg`
- `ENV` — build time **and** run time; override with `-e`
- `ARG` before `FROM` is only usable in `FROM` lines
- Neither is safe for secrets — both show in `history` / `inspect`

CONFIG INSTRUCTIONS IN USE

```
# syntax=docker/dockerfile:1
ARG PY_VERSION=3.13
FROM python:${PY_VERSION}-slim
# ARGs declared before FROM must be re-declared after it
ARG APP_VERSION=dev
ENV PYTHONUNBUFFERED=1 \
    APP_ENV=production
LABEL org.opencontainers.image.version=$APP_VERSION \
      org.opencontainers.image.source="https://github.com/acme/api"
RUN useradd --create-home --uid 10001 app
WORKDIR /home/app
COPY --chown=app:app . .
USER app
VOLUME /home/app/data
STOPSIGNAL SIGINT
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD ["python", "healthcheck.py"]
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

NOTES

- → `# syntax=` must be the first line; picks the Dockerfile frontend version
- → `ARG` defaults can be overridden: `--build-arg APP_VERSION=1.4`
- → `HEALTHCHECK` sets status shown in `docker ps` (healthy/unhealthy)
- → The check needs a tool in the image; here a script that exits 1 on failure
- → `USER` affects every later RUN, CMD and ENTRYPOINT

INSTRUCTION REFERENCE

INSTRUCTION	PURPOSE
`ENV KEY=val`	Default env var for build + containers
`ARG NAME=default`	Build-time variable
`USER uid[:gid]`	User for later steps + runtime
`LABEL k=v`	Metadata (OCI keys recommended)
`HEALTHCHECK NONE`	Disable an inherited check

INSTRUCTION REFERENCE (CONT.)

INSTRUCTION	PURPOSE
`VOLUME /path`	Mark path as a volume mount point
`SHELL ["bash","-c"]`	Shell used by shell-form RUN
`STOPSIGNAL SIGINT`	Signal `docker stop` sends first
`ONBUILD COPY . .`	Runs in images built FROM this one
`EXPOSE 8000/udp`	Document port + protocol

⚠ COMMON MISTAKES

- `ENV API_KEY=…` or `ARG TOKEN` for secrets — readable by anyone with the image; use build secrets (Sheet 17)
- Declaring `ARG` before `FROM` and using it later — it's out of scope; repeat `ARG NAME` after `FROM`
- `HEALTHCHECK CMD curl …` on an image without curl — always unhealthy
- `VOLUME` in a base image — later `RUN` changes to that path are thrown away

✓ TIPS & BEST PRACTICES

- Use OCI label keys (`org.opencontainers.image.*`) so registries display them
- Give the app user a fixed UID (e.g. 10001) so file permissions are predictable
- Add `--start-period` so slow-starting apps aren't marked unhealthy too early
- Keep `# syntax=docker/dockerfile:1` to get new features like `RUN --mount`

QUICK SUMMARY

- `ARG` — build only · `ENV` — build + run
- `USER` — drop root
- `HEALTHCHECK` — container health status
- `LABEL` / `STOPSIGNAL` / `SHELL` — metadata and behavior

docker build & BuildKit

BUILDING

`docker build` sends the **build context** (a folder, Git URL or tarball) to the builder and runs the Dockerfile against it. Since Engine 23 the builder is **BuildKit**: it runs independent steps in parallel, skips stages you don't need, and supports cache and secret mounts. `docker build` is an alias for `docker buildx build` with the default builder.

BUILD CONTEXT

- `.` at the end = context folder, not the Dockerfile
- Only files in the context can be `COPY`'d
- `.dockerignore` shrinks what's sent
- First line of output shows context size transferred

BUILD COMMANDS

```
docker build -t myapp:1.0 .                       # Dockerfile in current folder
docker build -t myapp:1.0 -t myapp:latest .       # several tags at once
docker build -f docker/Dockerfile.prod -t myapp:prod .   # another Dockerfile
docker build --build-arg APP_VERSION=1.0 -t myapp:1.0 .
docker build --target test -t myapp:test .        # stop at a stage
docker build --no-cache -t myapp:1.0 .            # ignore cache
docker build --pull -t myapp:1.0 .                # re-check the base image
docker build --progress=plain . 2>&1 | tee build.log   # full output
docker build --platform linux/amd64 -t myapp:amd64 .
docker build -t myapp https://github.com/acme/app.git#main   # Git context
docker build - < Dockerfile                       # no context at all
docker build --secret id=npm,src=$HOME/.npmrc -t myapp .
docker builder prune                              # clear build cache
```

CODE EXPLAINED

- → `-t name:tag` — tag the result (repeatable)
- → `-f` — Dockerfile path; context is still the last argument
- → `--target` — build only up to a named stage
- → `--pull` — always fetch the newest base image for that tag
- → `--progress=plain` — show every RUN line (great for debugging)
- → `--secret` — mount a secret into one RUN step (Sheet 17)

OTHER BUILD FLAGS

FLAG	DOES
`--load`	Put the result in the local image store
`--push`	Push straight to the registry
`--output type=local,dest=out`	Export files, not an image
`--label k=v`	Add a label
`--network none`	No network for RUN steps

ENVIRONMENT & DEBUGGING

ITEM	DOES
`DOCKER_BUILDKIT=0`	Use legacy builder (deprecated)
`BUILDKIT_PROGRESS=plain`	Default to plain output
`docker buildx history ls`	Past builds with status + time
`docker buildx du`	Build cache usage
`--check`	Lint the Dockerfile, don't build

⚠ COMMON MISTAKES

- `docker build -f Dockerfile` with no context — the trailing `.` (or a path) is required
- Sending a 2 GB context because `node_modules` or datasets aren't in `.dockerignore`
- `COPY ../shared /app` — paths outside the context are invisible; move the context up a level and use `-f`
- Using `--no-cache` to fix every problem — fix the step order instead; it's much slower

✓ TIPS & BEST PRACTICES

- Use `--progress=plain` when a RUN step fails; the default view hides most output
- `docker build --check .` catches common Dockerfile mistakes in seconds
- Tag with both a version and the commit SHA in CI
- Use `--build-context name=../shared` to add a second folder without widening the main context

QUICK SUMMARY

- `build -t name:tag .` — build + tag
- `-f` / `--target` — which file, which stage
- `--build-arg` / `--secret` — inputs
- `BuildKit` — default builder since Engine 23

Multi-Stage Builds & Layer Caching

FROM … AS · COPY --from= · layer order · RUN --mount=type=cache · --mount=type=secret · slim / alpine / distroless

SMALL IMAGES, FAST REBUILDS

A **multi-stage** build uses several `FROM` lines: early stages hold compilers and dev tools, and the final stage copies in only the finished output, so the shipped image stays small. **Layer caching** reuses a step when its instruction and inputs haven't changed; once one step misses, every step after it rebuilds.

CACHE RULES

- `COPY` / `ADD` miss when file contents change
- `RUN` misses only when its command text changes
- Order steps from least- to most-often changed
- Changing the base image invalidates everything

MULTI-STAGE GO BUILD WITH CACHE + SECRET

```
# syntax=docker/dockerfile:1
FROM golang:1.25 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 go build -o /out/api ./cmd/api

FROM build AS test
RUN go test ./...

FROM gcr.io/distroless/static-debian12 AS final
COPY --from=build /out/api /api
USER nonroot
ENTRYPOINT ["/api"]
```

CODE EXPLAINED

- → `AS build` — names a stage so others can refer to it
- → `--mount=type=cache` — keeps the module/compiler cache between builds, not in the image
- → `FROM build AS test` — stage built only with `--target test`
- → Final stage is distroless: no shell, no package manager, ~2 MB base
- → `COPY --from=build` — pull files from an earlier stage (or any image)

SECRETS AND OTHER MOUNTS

```
RUN --mount=type=secret,id=npm,target=/root/.npmrc npm ci
#   docker build --secret id=npm,src=$HOME/.npmrc .
RUN --mount=type=ssh git clone git@github.com:acme/priv.git
#   docker build --ssh default .
RUN --mount=type=bind,source=req.txt,target=/tmp/req.txt \
    pip install -r /tmp/req.txt
COPY --from=nginx:1.27 /etc/nginx/nginx.conf /etc/nginx/
```

CHOOSING A FINAL BASE

BASE	SIZE	NOTES
`ubuntu / debian`	~30–80 MB	Full tools; easy debugging
`*-slim`	~30–50 MB	Debian, fewer packages
`alpine`	~5 MB	musl libc; some wheels differ
`distroless`	~2–20 MB	No shell; use `docker debug`
`scratch`	0 MB	Static binaries only

⚠ COMMON MISTAKES

- `COPY . .` before installing dependencies — any code change re-runs the whole install
- `RUN apt-get update` in one step and `apt-get install` in another — stale cached index; combine them
- Passing tokens with `ARG` — stored in image history; use `--mount=type=secret`
- Shipping the `build` stage by forgetting a final `FROM` — image includes the whole toolchain

✓ TIPS & BEST PRACTICES

- Put `--target` stages (test, lint) after build so CI can run them on demand
- Use `apt-get install --no-install-recommends` and clean lists in the same RUN
- Name every stage; `COPY --from=0` breaks when stages move
- Check results with `docker image ls` and `docker image history`

QUICK SUMMARY

- `FROM … AS name` — define a stage
- `COPY --from=name` — take only the output
- `--mount=type=cache` — fast dependency installs
- `--mount=type=secret` — secrets that never land in layers

Buildx & Multi-Platform Builds

docker buildx create / use · --platform linux/amd64,linux/arm64 · --push · --cache-to / --cache-from · imagetools inspect · buildx bake

ONE TAG, MANY CPUS

Laptops are often **arm64** (Apple silicon) while servers are often **amd64**. A **multi-platform** image is one tag pointing at an index with a variant per architecture; each host pulls the one it needs. **Buildx** builds them using QEMU emulation, native nodes, or cross-compiling in the Dockerfile.

WHERE MULTI-PLATFORM WORKS

- Docker Desktop and Engine 29+ with the **containerd image store**
- Or a `docker-container` builder (`buildx create`)
- Emulation needs QEMU (`binfmt`); Desktop includes it
- Emulated builds are slow; cross-compile when you can

BUILDX COMMANDS

```
docker buildx ls                                  # builders + platforms
docker buildx create --name multi --driver docker-container --use
docker run --privileged --rm tonistiigi/binfmt --install all    # QEMU (Linux)

docker buildx build --platform linux/amd64,linux/arm64 \
  -t ghcr.io/acme/api:1.4 --push .

docker buildx build --platform linux/amd64 --load -t api:amd64 .

docker buildx build \
  --cache-from type=registry,ref=ghcr.io/acme/api:cache \
  --cache-to   type=registry,ref=ghcr.io/acme/api:cache,mode=max \
  -t ghcr.io/acme/api:1.4 --push .

docker buildx imagetools inspect ghcr.io/acme/api:1.4   # list platforms
docker buildx bake                        # build targets from docker-bake.hcl
```

NOTES

- → `--push` sends all platforms + the index in one go
- → `--load` puts the image in the local store (multi-platform needs the containerd store)
- → `mode=max` caches every stage, not just the final one
- → In GitHub Actions use `--cache-to type=gha`
- → `TARGETPLATFORM` , `TARGETARCH` , `BUILDPLATFORM` are automatic build args

CROSS-COMPILE INSTEAD OF EMULATING

```
FROM --platform=$BUILDPLATFORM golang:1.25 AS build
ARG TARGETOS TARGETARCH
RUN GOOS=$TARGETOS GOARCH=$TARGETARCH \
    go build -o /out/app .
FROM alpine:3.22
COPY --from=build /out/app /app
```

DOCKER-BAKE.HCL

```
group "default" { targets = ["api", "web"] }
target "api" {
  context   = "./api"
  platforms = ["linux/amd64", "linux/arm64"]
  tags      = ["ghcr.io/acme/api:1.4"]
}
target "web" { context = "./web" }
```

⚠ COMMON MISTAKES

- Running an arm64 image on amd64 servers — `exec format error` ; build for `linux/amd64` too
- `--platform a,b --load` on the classic image store — error; enable containerd or use `--push`
- Emulating a 20-minute compile under QEMU — can be 5–10× slower; cross-compile with `$BUILDPLATFORM`
- Forgetting `docker buildx use` after `create` — builds still go to the old builder

✓ TIPS & BEST PRACTICES

- Check what a tag supports with `buildx imagetools inspect` before deploying
- Use a registry or GHA cache in CI so builds don't start cold
- Bake files keep long build commands in version control
- `docker buildx bake --print` shows the resolved targets without building

QUICK SUMMARY

- `--platform a,b` — multi-arch build
- `--push` / `--load` — where the result goes
- `--cache-to/--cache-from` — shared cache
- `$BUILDPLATFORM` + `$TARGETARCH` — cross-compile

Volumes & Bind Mounts

volume create / ls / rm · -v vs --mount · named vs anonymous · bind mounts · :ro · tmpfs · volume backup pattern

KEEPING DATA

A container's writable layer disappears with `docker rm`, so anything worth keeping goes in a mount. **Volumes** are storage managed by Docker (under `/var/lib/docker/volumes`) and are the right choice for databases and app data. **Bind mounts** map a host path into the container, ideal for live-editing source code. **tmpfs** mounts live in memory only.

WHICH ONE?

- **Named volume** — databases, uploads, anything persistent
- **Bind mount** — dev source code, config files from the host
- **tmpfs** — secrets or scratch files that must never hit disk
- **Anonymous volume** — from `VOLUME` / `-v /path`; easy to lose

MOUNTING

```
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:17
docker run -d --name db \
  --mount type=volume,src=pgdata,dst=/var/lib/postgresql/data postgres:17

docker run --rm -v "$PWD":/app -w /app node:22 npm test      # bind mount
docker run --rm -v "$PWD/nginx.conf":/etc/nginx/nginx.conf:ro nginx
docker run --rm --mount type=bind,src="$PWD",dst=/app,readonly alpine ls /app
docker run --rm --tmpfs /scratch:size=100m alpine df -h /scratch

docker volume ls                        # all volumes
docker volume inspect pgdata            # Mountpoint, driver, labels
docker volume rm pgdata
docker volume prune -a                  # remove ALL unused volumes
docker rm -v web                        # + its anonymous volumes
```

-V VS --MOUNT

- → `-v src:dst[:opts]` — short; if `src` has no `/`, it's a volume name
- → `--mount type=…,src=…,dst=…` — explicit; preferred in scripts
- → `-v` creates a missing host path as a folder; `--mount` errors instead
- → `:ro` / `readonly` — mount read-only
- → An empty **volume** is pre-filled from the image's files at that path; a bind mount hides them

BACKUP AND RESTORE A VOLUME

```
docker run --rm -v pgdata:/data -v "$PWD":/backup alpine \
  tar czf /backup/pgdata.tgz -C /data .
docker run --rm -v pgdata:/data -v "$PWD":/backup alpine \
  tar xzf /backup/pgdata.tgz -C /data
```

MOUNT OPTIONS

OPTION	EFFECT
`:ro / readonly`	Read-only in the container
`:z / :Z`	SELinux relabel (shared / private)
`volume-opt=…`	Driver options (NFS, CIFS)
`tmpfs-size=64m`	Cap a tmpfs mount
`volume-subpath=dir`	Mount one folder of a volume

⚠ COMMON MISTAKES

- `docker volume prune -a` thinking it's safe — it deletes every volume no container uses right now
- Bind-mounting over the app folder and losing `node_modules` from the image — add a volume for that subfolder
- Mixing up `-v data:/x` (volume) and `-v ./data:/x` (bind) — a leading `./` or `/` makes it a host path
- Permission denied on bind mounts — the container UID doesn't own the host files; match UIDs or use `--user`

✓ TIPS & BEST PRACTICES

- Name volumes explicitly; anonymous ones get random IDs and are easy to delete by accident
- Back up volumes with a throwaway container + `tar`, as shown above
- Use `:ro` for config files so a compromised app can't rewrite them
- On macOS/Windows Desktop, named volumes are much faster than bind mounts for heavy I/O

QUICK SUMMARY

- `-v name:/path` — named volume
- `-v $PWD:/path` — bind mount
- `--tmpfs /path` — RAM only
- `volume ls/inspect/rm/prune` — manage

Networking

-p host:container · network create · bridge / host / none · user-defined bridge DNS · --network · network connect · overlay / macvlan

HOW CONTAINERS TALK

Each container gets its own network stack. On the default **bridge** network, containers can reach each other only by IP. Create your own **user-defined bridge** and containers on it find each other **by name** through Docker's built-in DNS. To reach a container from outside the host, **publish** a port with –p.

NOTES

- `-p 8080:80` = host port 8080 → container port 80
- Containers on the same network use the **container** port, not the published one
- Published ports bypass `ufw` -style firewalls on Linux; bind to `127.0.0.1` to keep them private

NETWORKS IN PRACTICE

```
docker network create appnet                    # user-defined bridge
docker run -d --name db  --network appnet -e POSTGRES_PASSWORD=dev postgres:17
docker run -d --name api --network appnet -p 8080:3000 \
  -e DATABASE_URL=postgres://postgres:dev@db:5432/postgres myapi
docker exec api ping -c1 db                     # name resolves via DNS

docker network connect frontend api             # attach a 2nd network
docker network disconnect frontend api
docker run -d --network-alias search --network appnet elasticsearch:8.15

docker network ls
docker network inspect appnet -f '{{range .Containers}}{{.Name}} {{end}}'
docker network rm appnet
docker network prune                            # unused networks
```

CODE EXPLAINED

→ `db` resolves to the db container's IP for anything on `appnet`

→ API listens on 3000 inside; reachable at `localhost:8080` from the host

→ A container can join several networks (e.g. frontend + backend)

→ `--network-alias` adds an extra DNS name; several containers can share one

PUBLISHING PORTS

FLAG	RESULT
`-p 8080:80`	All host IPs, port 8080 → 80
`-p 127.0.0.1:8080:80`	Localhost only
`-p 8080:80/udp`	UDP
`-p 80`	Random host port → 80
`-P`	All EXPOSEd ports, random
`-p 9000-9002:9000-9002`	Port range

NETWORK DRIVERS

DRIVER	USE
`bridge`	Default; single host, NAT
`host`	Share host's network (no -p needed)
`none`	No networking at all
`overlay`	Span Swarm nodes (Sheet 27)
`macvlan`	Own MAC/IP on the physical LAN
`ipvlan`	Like macvlan, shared MAC

⚠ COMMON MISTAKES

- Using `localhost` to reach another container — that's the container itself; use the other container's name
- Relying on the default `bridge` network for names — DNS only works on user-defined networks
- Connecting to `db:5433` because you published `-p 5433:5432` — inside the network it's still `5432`
- Publishing database ports on `0.0.0.0` on a public server — bind to `127.0.0.1` or don't publish

✓ TIPS & BEST PRACTICES

- Create one network per app (Compose does this for you)
- Use `docker run --rm --network appnet nicolaka/netshoot` for `dig`, `curl`, `tcpdump`
- `--network container:api` shares another container's network stack (sidecars, debugging)
- `host` networking works fully on Linux; on Desktop it is limited

QUICK SUMMARY

- `-p host:container` — publish a port
- `network create` — user-defined bridge with DNS
- `--network name` — join at start
- `network connect` — join later

Compose Basics

DOCKER COMPOSE

Compose describes a multi-container app in one YAML file: each **service** becomes one or more containers, and Compose creates a shared network so services reach each other by service name. `docker compose up` builds, creates and starts everything; `down` tears it all down. The project name (default: the folder name) prefixes every container, network and volume.

VERSIONS

- Compose v2 is a CLI plugin: `docker compose` (space)
- **Legacy** v1 `docker-compose` (hyphen) is end-of-life
- The top-level `version:` key is obsolete and ignored
- Default file names: `compose.yaml` , then `docker-compose.yml`

COMPOSE.YAML

```yaml
services:
  web:
    build: .                      # build from ./Dockerfile
    ports: ["8080:3000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
    depends_on: [db]
  db:
    image: postgres:17            # use a ready-made image
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

EVERYDAY COMMANDS

```
docker compose up -d           # create + start all
docker compose up -d --build   # rebuild first
docker compose ps              # this project's containers
docker compose logs -f web     # follow one service
docker compose stop            # stop, keep containers
docker compose start
docker compose restart web
docker compose down            # remove containers + network
docker compose down -v         # …and named volumes
docker compose pull            # newer images
docker init                    # generate starter files
```

IMAGE VS BUILD

KEY	MEANING
`image: postgres:17`	Pull and run this image
`build: .`	Build from ./Dockerfile
`build + image`	Build, then tag with that name
`build.context`	Folder sent to the builder
`build.dockerfile`	Alternate Dockerfile path
`build.args`	Build args (ARG)

PROJECT NAMING

ITEM	NAME
Project	folder name or `-p name` / `name:`
Container	`<project>-web-1`
Network	`<project>_default`
Volume	`<project>_pgdata`
Service DNS	`web`, `db`

⚠ COMMON MISTAKES

- Using `localhost` in `DATABASE_URL` — use the service name (`db`) from inside the network
- `docker compose down -v` in a real environment — deletes the database volume
- Editing code and running `up -d` — images aren't rebuilt unless you add `--build` (or use `watch`)
- Two copies of the same folder name on one host — same project name, so they collide; set `-p`

✓ TIPS & BEST PRACTICES

- Quote port mappings (`"8080:3000"`) so YAML doesn't misread values like `22:22`
- Run `docker compose config` to see the final merged file with variables filled in
- `docker compose up -d web` starts one service plus whatever it depends on
- `docker init` writes a Dockerfile, `compose.yaml` and `.dockerignore` for common languages

QUICK SUMMARY

- `compose.yaml` — services, volumes, networks
- `up -d` / `down` — start / remove the stack
- `logs -f` / `ps` — watch the stack
- service name = DNS name

Compose Service Configuration

ports · environment / env_file · command / entrypoint · restart · healthcheck · depends_on: condition · deploy.resources

SERVICE KEYS

Almost every `docker run` flag has a matching key under a service. The most useful extra is `depends_on` with **conditions**: combined with a `healthcheck`, Compose waits until a dependency is actually ready (not just started) before starting the services that need it.

MORE KEYS → DOCKER RUN

COMPOSE KEY	DOCKER RUN
`container_name`	`--name`
`user`	`--user`
`working_dir`	`-w`
`read_only`	`--read-only`
`cap_drop`	`--cap-drop`
`extra_hosts`	`--add-host`

A FULLY CONFIGURED SERVICE

```yaml
services:
  api:
    build: ./api
    image: acme/api:dev
    ports: ["127.0.0.1:8080:3000"]
    env_file: [.env, .env.local]
    environment:
      NODE_ENV: development
      LOG_LEVEL: ${LOG_LEVEL:-info}
    command: ["npm", "run", "dev"]
    restart: unless-stopped
    init: true
    depends_on:
      db: { condition: service_healthy }
      migrate: { condition: service_completed_successfully }
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 10s
      retries: 5
      start_period: 15s
    deploy:
      resources: { limits: { cpus: "1.5", memory: 512M } }
  db:
    image: postgres:17
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
```

CODE EXPLAINED

- → `127.0.0.1:` — publish on localhost only
- → `${LOG_LEVEL:-info}` — from shell/ `.env` , default `info`
- → `service_healthy` — wait for db's healthcheck to pass
- → `service_completed_successfully` — wait for a one-shot job (migrations) to exit 0
- → `service_started` — the default: just started
- → `CMD` runs exec form; `CMD-SHELL` runs through `sh -c`
- → `init: true` — same as `--init`
- → `limits` — same as `--cpus` / `--memory` ; `environment` beats `env_file`

⚠ COMMON MISTAKES

- Plain `depends_on: [db]` — only waits for the container to **start**, not for Postgres to accept connections
- `healthcheck` using `curl` in an image without it — service never becomes healthy
- Setting `container_name` — prevents scaling and clashes across projects
- Unquoted `cpus: 1.5` or `"8080:80"` mistakes — quote numbers-with-colons and decimals

✓ TIPS & BEST PRACTICES

- Pair every `depends_on` you care about with a `healthcheck` on the dependency
- Run migrations as a one-shot service and gate the app on `service_completed_successfully`
- Use `restart: unless-stopped` for anything that should come back after a reboot
- Short env syntax `- LOG_LEVEL` passes the value straight from your shell

QUICK SUMMARY

- `ports` / `environment` / `env_file` — basics
- `depends_on.condition` — real startup order
- `healthcheck` — readiness probe
- `deploy.resources.limits` — CPU/memory caps

Compose Volumes, Networks & Secrets

SHARED RESOURCES

Top-level `volumes:`, `networks:`, `secrets:` and `configs:` declare resources once; services then reference them by name. Compose creates them with the project prefix unless marked `external: true`. Variables like `${TAG}` are filled in from your shell or a `.env` file next to `compose.yaml` before anything runs, and **profiles** let optional services stay off by default.

NOTES

- Secrets mount as files at `/run/secrets/<name>`
- `.env` feeds **interpolation**; `env_file` feeds containers
- Services with no `profiles` always start

COMPOSE.YAML

```yaml
services:
  api:
    image: acme/api:${TAG:-latest}
    networks: [backend, frontend]
    volumes: [uploads:/app/uploads, ./config:/app/config:ro]
    secrets: [db_password]
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password
  db:
    image: postgres:17
    networks: [backend]
    volumes: [pgdata:/var/lib/postgresql/data]
  adminer: { image: adminer, profiles: [debug], ports: ["8081:8080"] }
volumes:
  pgdata:
  uploads: { external: true }       # Compose won't create/delete
networks:
  backend: { internal: true }       # no outside access
  frontend:
secrets:
  db_password: { file: ./secrets/db_password.txt }
```

.ENV (INTERPOLATION)

```
TAG=1.4.2
COMPOSE_PROJECT_NAME=shop
COMPOSE_PROFILES=debug
```

NOTES

→ `profiles: [debug]` — start with `--profile debug`

→ `*_FILE` env vars: many official images read secrets this way

INTERPOLATION SYNTAX

SYNTAX	RESULT
`${VAR}`	Value, or empty + warning
`${VAR:-default}`	Default if unset or empty
`${VAR:?error msg}`	Fail if unset or empty
`$$`	Literal $

PROFILES & EXTERNALS

COMMAND	DOES
`docker compose --profile debug up`	Include debug services
`COMPOSE_PROFILES=a,b`	Same via env / .env
`docker compose up adminer`	Naming a service enables it
`docker volume create uploads`	Create an external volume

⚠ COMMON MISTAKES

- Expecting `.env` values inside containers — they're only for `${…}` substitution unless you also use `env_file`
- Committing `secrets/*.txt` or `.env` to git — add them to `.gitignore`
- `docker compose down -v` with `external` volumes — those survive, but project volumes are deleted
- Writing `$PATH`-style shell vars in `command:` — Compose substitutes them; escape as `$$PATH`

✓ TIPS & BEST PRACTICES

- Use `${VAR:?set VAR first}` for values that must be provided
- Put debug tools (adminer, mailhog) behind a profile
- Mark backend networks `internal: true` so databases have no internet access
- `docker compose config` shows the file after interpolation — great for debugging

QUICK SUMMARY

- `volumes` / `networks` — declared once, shared
- `secrets` — files at `/run/secrets/…`
- `${VAR:-default}` — fill from shell or `.env`
- `profiles` — optional services

Compose Workflows

compose exec vs run --rm · build --pull · watch / develop.watch · -f override files · compose.override.yaml · config · include · --scale

WORKING WITH A STACK DAY TO DAY

Beyond `up` and `down`, Compose has commands for one-off tasks, live code sync during development, and layering files for different environments. Compose automatically merges `compose.override.yaml` on top of `compose.yaml`, which is the usual place for dev-only settings.

EXEC VS RUN

- `exec` — command in the **running** service container
- `run --rm` — new one-off container from the service config
- `run` skips `ports` unless you add `--service-ports`
- `run` starts dependencies first

COMMANDS

```
docker compose exec web sh                          # shell in running container
docker compose run --rm web npm test                # one-off container
docker compose run --rm --no-deps web rake db:migrate

docker compose build --pull web                     # rebuild with fresh base
docker compose up -d --force-recreate web
docker compose up -d --scale worker=3               # 3 worker containers
docker compose watch                                # sync + rebuild on change

docker compose -f compose.yaml -f compose.prod.yaml up -d
docker compose config                               # final merged YAML
docker compose ls                                   # all projects on host
```

NOTES

- → `--no-deps` — don't start linked services
- → `--force-recreate` — new containers even if config is unchanged
- → `--scale` fails if the service has `container_name` or a fixed host port
- → Later `-f` files override earlier ones key by key
- → Lists like `ports` are **merged**, not replaced

DEVELOP.WATCH

```
services:
  web:
    build: .
    develop:
      watch:
        - action: sync
          path: ./src
          target: /app/src
        - action: sync+restart
          path: ./config
          target: /app/config
        - action: rebuild
          path: package.json
```

INCLUDE + OVERRIDE

```
# compose.yaml
include:
  - ./db/compose.yaml      # own project dir
services:
  web:
    build: .

# compose.override.yaml (auto-loaded)
services:
  web:
    volumes: ["./src:/app/src"]
    environment: { DEBUG: "1" }
```

⚠ COMMON MISTAKES

- Running tests with `exec` and finding the dev server's state — use `run --rm` for a clean container
- Prod deploy picking up `compose.override.yaml` — pass `-f compose.yaml -f compose.prod.yaml` explicitly
- `watch` with a bind mount of the same folder — files sync twice; use one or the other
- Expecting `--scale` to load-balance published ports — use a proxy (Traefik, nginx) in front

✓ TIPS & BEST PRACTICES

- `sync` for interpreted code, `sync+restart` for config, `rebuild` for dependency files
- Use `docker compose config > resolved.yaml` to review exactly what will deploy
- `include` lets each team own its own Compose file while sharing one stack
- `docker compose ls` finds stacks you forgot were running

QUICK SUMMARY

- `exec` vs `run --rm` — existing vs fresh container
- `watch` — live sync / rebuild
- `-f a -f b` / override file — layer configs
- `config` — print the final merged model

Swarm Setup & Nodes

docker swarm init · swarm join · join-token · manager vs worker · node ls / promote / demote · node update --availability drain · Raft quorum

SWARM MODE

Swarm mode is Docker's built-in orchestrator: it joins several Docker hosts into one cluster and keeps **services** at the state you declare, rescheduling containers when a node fails. **Managers** store cluster state with the Raft consensus algorithm and schedule work; **workers** just run tasks. It ships with Docker Engine, so there's nothing extra to install.

PORTS TO OPEN BETWEEN NODES

- `2377/tcp` — cluster management (managers)
- `7946/tcp+udp` — node discovery (gossip)
- `4789/udp` — overlay network traffic (VXLAN)
- Managers are also workers unless drained

BUILDING A CLUSTER

```
# On the first manager
docker swarm init --advertise-addr 10.0.0.10
docker swarm join-token worker          # print the worker join command
docker swarm join-token manager         # …and the manager one

# On each other node (command printed by join-token)
docker swarm join --token SWMTKN-1-<token> 10.0.0.10:2377

# Back on a manager
docker node ls                          # * = the node you're on
docker node promote node2 node3         # 3 managers for HA
docker node update --availability drain node4   # move tasks off
docker node update --availability active node4
docker node update --label-add zone=east node4
docker node inspect node4 --pretty
docker swarm join-token --rotate worker # invalidate leaked tokens
docker swarm leave                      # on a worker; then `node rm` it
```

CODE EXPLAINED

- → `--advertise-addr` — IP other nodes use to reach this one
- → `drain` — no new tasks; existing ones move elsewhere (maintenance)
- → `pause` — keep running tasks, schedule nothing new
- → Node labels drive placement constraints (Sheet 26)
- → `swarm leave --force` on the last manager destroys the swarm

MANAGERS & QUORUM

MANAGERS	CAN LOSE	NOTES
1	0	Fine for dev; no HA
3	1	Smallest HA setup
5	2	Common for production
7	3	Practical maximum
even (2, 4)	same as one fewer	Adds risk, no benefit

CLUSTER COMMANDS

COMMAND	DOES
`docker info | grep Swarm`	Is this node in a swarm?
`docker swarm init --autolock`	Encrypt Raft keys at rest
`docker swarm unlock`	Unlock after manager restart
`docker node demote node3`	Manager → worker
`docker node rm node5`	Remove a node that has left

⚠ COMMON MISTAKES

- Running 2 managers — losing either one loses quorum; use an odd number
- Firewalls blocking `4789/udp` or `7946` — services deploy but containers on different nodes can't talk
- Treating the join token like a config value — anyone with it can join; rotate it if it leaks
- Taking a node down without draining — tasks restart elsewhere only after it's marked down

✓ TIPS & BEST PRACTICES

- Keep managers small and dedicated in production: `drain` them so they only manage
- Spread managers across failure zones (racks, availability zones)
- Back up `/var/lib/docker/swarm` on a manager to restore the cluster state
- Need autoscaling, a big ecosystem or managed cloud offerings? Use Kubernetes instead

QUICK SUMMARY

- `swarm init` / `join` — create and grow a cluster
- `node promote/demote` — manager vs worker
- `node update --availability drain` — maintenance
- odd managers — (N-1)/2 failures tolerated

Swarm Services & Rolling Updates

docker service create / ls / ps · service scale · replicated vs global · --update-parallelism · --update-delay · service rollback · --constraint

SERVICES AND TASKS

A **service** is the desired state: image, replica count, ports, limits. Swarm turns it into **tasks**, each running one container on some node, and replaces tasks that fail. Changing a service (new image, env, limits) triggers a **rolling update** that replaces tasks a few at a time and can roll back automatically on failure.

SERVICE MODES

- **replicated** — run N copies anywhere (default)
- **global** — exactly one per node (agents, log shippers)
- Service name = DNS name on its overlay networks
- `service create` flags mirror `docker run` flags

CREATE, SCALE, UPDATE

```
docker service create --name web --replicas 3 -p 8080:80 \
  --limit-memory 256M --constraint node.role==worker nginx:1.27
docker service ls                          # replicas: 3/3
docker service ps web                      # tasks + which node they're on
docker service logs -f web                 # logs from all tasks
docker service scale web=6
docker service update \
  --image nginx:1.28 \
  --update-parallelism 2 --update-delay 10s \
  --update-order start-first \
  --update-failure-action rollback web
docker service rollback web                # back to previous spec

docker service create --mode global --name agent \
  --mount type=bind,src=/var/log,dst=/host/log,ro datadog/agent
docker service rm web
```

CODE EXPLAINED

- → `--constraint` — only schedule where the expression is true
- → `--update-parallelism 2` — replace 2 tasks at a time
- → `--update-delay 10s` — pause between batches
- → `start-first` — start new task before stopping old (no gap)
- → `--update-failure-action rollback` — undo automatically on failure
- → `--mode global` — one task per node

PLACEMENT CONSTRAINTS

EXPRESSION	MATCHES
`node.role==manager`	Manager nodes
`node.labels.zone==east`	Nodes with that label
`node.platform.arch==aarch64`	ARM nodes
`--placement-pref spread=node.labels.zone`	Spread across zones

OTHER UPDATE FLAGS

FLAG	DOES
`--env-add K=V / --env-rm K`	Change env vars
`--publish-add 9090:90`	Add a published port
`--replicas-max-per-node 1`	Spread replicas
`--health-cmd / --health-interval`	Override HEALTHCHECK

⚠ COMMON MISTAKES

- Updating with `--image nginx:latest` when the tag hasn't changed — Swarm sees no change; use versioned tags
- No HEALTHCHECK — Swarm considers a task healthy as soon as it starts, so bad releases roll out fully
- Constraints no node satisfies — tasks sit in `Pending`; check `docker service ps --no-trunc web`
- Expecting `docker ps` on a manager to show all containers — it shows only that node; use `service ps`

✓ TIPS & BEST PRACTICES

- Always set `--update-failure-action rollback` plus a healthcheck for safe deploys
- `start-first` gives zero-downtime updates if you have spare capacity
- `docker service ps web --filter desired-state=running` hides old, shut-down tasks
- Prefer stacks (Sheet 27) over long `service create` commands

QUICK SUMMARY

- `service create/scale/rm` — manage services
- `service ps` / `logs` — tasks across nodes
- `service update --image` — rolling update
- `service rollback` — previous version

Stacks, Secrets & Overlay Networks

STACKS

A **stack** deploys a whole Compose file to a swarm: each service becomes a swarm service, networks become **overlay** networks spanning all nodes, and secrets and configs are stored encrypted in the Raft log. The same `compose.yaml` format works (Sheets 21–23), but the `deploy:` section now matters and some build-time keys are ignored.

COMPOSE VS STACK

- `build:` is ignored — push images to a registry first
- `depends_on` , `container_name` are ignored
- `.env` is **not** read; export variables in your shell
- Re-run `stack deploy` to apply changes

STACK.YAML

```yaml
services:
  web:
    image: ghcr.io/acme/web:1.4.2
    ports: ["80:8080"]
    networks: [front]
    secrets: [db_password]
    deploy:
      replicas: 3
      placement:
        constraints: [node.role == worker]
      update_config: { parallelism: 1, delay: 10s, order: start-first,
                       failure_action: rollback }
      restart_policy: { condition: on-failure, max_attempts: 3 }
      resources: { limits: { cpus: "0.5", memory: 256M } }
networks:
  front: { driver: overlay }
secrets:
  db_password: { external: true }
```

COMMANDS

```
printf 's3cret' |
  docker secret create db_password -
docker secret ls
docker config create web_conf app.yaml
docker network create -d overlay \
  --attachable --opt encrypted net1

docker stack deploy -c stack.yaml shop
docker stack ls
docker stack services shop
docker stack ps shop
docker stack rm shop
```

ROUTING MESH & DISCOVERY

FEATURE	BEHAVIOR
`ports: ["80:8080"]`	Port 80 open on **every** node, load-balanced to tasks
`mode: host (long syntax)`	Publish only on nodes running a task
`web (DNS)`	Virtual IP that balances across tasks
`tasks.web (DNS)`	All task IPs (for client-side balancing)
`endpoint_mode: dnsrr`	DNS round-robin instead of VIP

SECRETS & CONFIGS

FACT	DETAIL
Mount path	`/run/secrets/<name>` (in-memory)
Configs	Any path via `target`; not encrypted at rest
Immutable	Can't edit; create `db_password_v2`, update
`--attachable`	Lets `docker run` join the overlay

⚠ COMMON MISTAKES

- Deploying a file with `build:` only — no image on other nodes; build, push, then reference the tag
- Expecting `.env` substitution — `docker stack deploy` doesn't read it; `export` vars or pre-render with `docker compose config`
- Using a named local volume for a database with 3 replicas — each node gets its own empty volume
- Trying to `docker secret rm` a secret in use — remove it from services first

✓ TIPS & BEST PRACTICES

- Version secrets and configs in their names (`_v2`) to rotate without downtime
- Pin a stateful service to one node with a label constraint, or use a shared storage driver
- Add `--opt encrypted` for overlay networks carrying sensitive traffic
- Re-run `stack deploy` to apply changes; Swarm updates only what changed

QUICK SUMMARY

- `stack deploy -c file name` — deploy/update
- `deploy:` — replicas, placement, updates
- `secret create` — encrypted, mounted as files
- `overlay` + routing mesh — multi-node networking

Cleanup & Troubleshooting

system df · system prune -a --volumes · builder prune · logging drivers / log-opts · port already allocated · socket permission denied · no space left

KEEPING DOCKER HEALTHY

Docker never deletes images, stopped containers, volumes or build cache on its own, so disks fill up over time. `docker system df` shows where the space went and the `prune` commands reclaim it. Most other problems come down to a handful of error messages, listed below with their usual fixes.

WHAT PRUNE REMOVES

- `container prune` — stopped containers
- `image prune` — dangling images (`-a` : all unused)
- `volume prune` — unused anonymous volumes (`-a` : named too)
- `builder prune` — build cache

RECLAIMING SPACE

```
docker system df                        # images, containers, volumes, cache
docker system df -v                     # per-object detail
docker system prune                     # stopped containers, dangling images,
                                        # unused networks, dangling build cache
docker system prune -a                  # + all images not used by a container
docker system prune -a --volumes        # + unused volumes (data loss!)
docker image prune -a --filter "until=168h"   # unused and older than 7 days
docker builder prune --filter until=72h       # cache older than 3 days

# Where the space lives
sudo du -sh /var/lib/docker/*           # Linux Engine
```

NOTES

- → `-f` skips the confirmation prompt (careful in scripts)
- → `--filter until=` and `label=` work on most prune commands
- → Desktop: the VM disk doesn't shrink automatically; prune, then check Settings
- → Rotate logs: `--log-driver local` or `--log-opt max-size=10m`

COMMON ERRORS

MESSAGE	CAUSE → FIX
`port is already allocated`	Something uses the host port → change `-p` or stop it (`lsof -i :8080`)
`permission denied …docker.sock`	User not in `docker` group → `usermod -aG docker $USER`, re-login
`Cannot connect to the Docker daemon`	Daemon/Desktop not running or wrong context/`DOCKER_HOST`
`no space left on device`	Disk full → `system df`, prune, raise Desktop disk limit
`exec format error`	Wrong CPU architecture → `--platform` or multi-arch build
`pull access denied / not found`	Typo, private repo or not logged in → `docker login`
`toomanyrequests`	Docker Hub rate limit → log in or use a mirror
`OCI runtime create failed`	Bad entrypoint/mount path → read the full message

⚠ COMMON MISTAKES

- `docker system prune -a --volumes` on a server — deletes database volumes not attached to a running container
- Fixing socket errors with `sudo chmod 666 /var/run/docker.sock` — gives every user root access
- Leaving `json-file` unrotated — a chatty container can fill the disk
- Restarting Docker to clear an error without reading it — the message usually names the exact cause

✓ TIPS & BEST PRACTICES

- Schedule `docker image prune -a --filter until=168h -f` on CI runners
- Set default log rotation in `daemon.json` for every container (Sheet 03)
- `docker events` + `docker logs` + `docker inspect` answer most "why did it stop?" questions
- `journalctl -u docker` shows daemon errors on Linux; `docker desktop logs` on Desktop

QUICK SUMMARY

- `system df` — see usage
- `system prune [-a] [--volumes]` — reclaim
- `builder prune` — clear build cache
- `--log-opt max-size` — rotate logs

Security & Best Practices

non-root USER · --cap-drop ALL · --security-opt no-new-privileges · secrets not ENV · pin by digest · docker scout cves · SBOM · minimal base images

DEFENSE IN DEPTH

Containers share the host kernel, so a container running as root with extra privileges is only a short step from root on the host. Good practice layers several controls: small trusted base images, non-root users, dropped capabilities, read-only filesystems, secrets kept out of images, and regular vulnerability scans.

NEVER DO

- `--privileged` for normal apps (full host device access)
- Mount `/var/run/docker.sock` into untrusted containers
- Bake passwords into `ENV`, `ARG` or `COPY`'d files
- Run unpinned `latest` images in production

A LOCKED-DOWN RUN

```
docker run -d --name api \
  --user 10001:10001 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL --cap-add NET_BIND_SERVICE \
  --security-opt no-new-privileges \
  --memory 512m --cpus 1 --pids-limit 100 \
  -p 127.0.0.1:8080:8080 \
  ghcr.io/acme/api:1.4.2@sha256:<digest>

docker scout quickview ghcr.io/acme/api:1.4.2     # summary of CVEs
docker scout cves --only-severity critical,high ghcr.io/acme/api:1.4.2
docker scout recommendations ghcr.io/acme/api:1.4.2   # better base images
docker buildx build --sbom=true --provenance=mode=max \
  -t ghcr.io/acme/api:1.4.2 --push .
```

CODE EXPLAINED

- `--user` — non-root UID (also set `USER` in the Dockerfile)
- `--cap-drop ALL` then add back only what's needed
- `no-new-privileges` — blocks setuid binaries from gaining root
- `@sha256:` — run exactly the image you scanned
- `--sbom` / `--provenance` — attach a package list and build record to the image

IMAGE CHECKLIST

PRACTICE	HOW
Minimal base	`-slim`, `alpine`, distroless, hardened images
Multi-stage	No compilers in the final image
Non-root	`USER 10001` in Dockerfile
No secrets	`RUN --mount=type=secret`
Rebuild often	`--pull` to pick up base patches
Scan in CI	`docker scout cves --exit-code`

RUNTIME CHECKLIST

PRACTICE	HOW
Drop capabilities	`--cap-drop ALL`
Read-only root	`--read-only` + `--tmpfs`
Limit resources	`--memory`, `--pids-limit`
Private ports	`-p 127.0.0.1:…`
Rootless daemon	`dockerd-rootless-setuptool.sh`
Isolate (Desktop)	Enhanced Container Isolation

⚠ COMMON MISTAKES

- Adding `--privileged` to "fix" a permission error — find the one capability or device actually needed
- Scanning once and never again — new CVEs appear daily; scan on every build and on a schedule
- Deleting a secret in a later layer (`RUN rm .env`) — it still exists in the earlier layer
- Trusting any image on Docker Hub — prefer Docker Official Images and Verified Publishers

✓ TIPS & BEST PRACTICES

- Add `--exit-code` to `docker scout cves` in CI to fail builds with critical CVEs
- Rebuild images weekly with `--pull` even when your code hasn't changed
- Use read-only bind mounts (`:ro`) for config files
- Run Docker Bench for Security or similar checks against hosts

QUICK SUMMARY

- `USER` / `--user` — never root
- `--cap-drop ALL` + `no-new-privileges`
- `@sha256:` — pin what you scanned
- `docker scout cves` — find and fix vulnerabilities

Quick Reference — CLI Commands

CONTAINERS

COMMAND	DOES	#
`docker run -d --name n img`	Start in background	07
`docker run --rm -it img sh`	Throwaway shell	07
`docker ps [-a]`	List running [all]	08
`docker start|stop|restart n`	Lifecycle	08
`docker kill [-s SIG] n`	Send signal / SIGKILL	08
`docker rm -f n`	Stop + remove	08
`docker update --memory 1g n`	Change limits live	11
`docker wait n`	Block, print exit code	08
`docker create img`	Create without starting	07
`docker pause|unpause n`	Freeze / thaw	08
`docker container prune`	Remove all stopped	08

INSPECT & DEBUG

COMMAND	DOES	#
`docker logs -f --tail 50 n`	Follow last 50 lines	09
`docker inspect -f '{{.State.Status}}' n`	One field	09
`docker exec -it n sh`	Shell in running container	10
`docker cp n:/path ./`	Copy out (or in)	10
`docker debug n`	Toolbox shell (Desktop)	10
`docker stats / top n`	Resources / processes	09
`docker port n · diff n`	Ports · changed files	09
`docker events`	Live daemon events	09
`docker logs --since 10m n`	Recent logs only	09
`docker exec -u root n sh`	Shell as root	10
`docker commit n img:dbg`	Snapshot for debugging	10

DOCKER RUN FLAGS

COMMAND	DOES	#
`-p 127.0.0.1:8080:80`	Publish port (localhost)	20
`-v vol:/data · -v $PWD:/app:ro`	Volume · bind mount	19
`-e K=V · --env-file .env`	Environment	07
`--network net · --network-alias a`	Network + DNS alias	20
`--restart unless-stopped`	Restart policy	08
`--memory 512m --cpus 1.5`	Resource limits	11
`--user 1000 --read-only --init`	Hardening	11
`--cap-drop ALL`	Drop capabilities	29
`-d · -it · --rm`	Detach · interactive · auto-remove	07
`--name n · -w /app`	Name · working dir	07
`--platform linux/amd64`	Pick architecture	07

IMAGES, BUILD & REGISTRY

COMMAND	DOES	#
`docker pull img:tag · @sha256:…`	Download	12
`docker image ls · rmi · history`	List · remove · layers	12
`docker save -o f.tar · load -i`	Images as tar	12
`docker tag src user/repo:1.0`	Add a name	13
`docker login · push`	Authenticate · upload	13
`docker build -t n:t -f F --target s .`	Build	16
`buildx build --platform a,b --push`	Multi-arch build	18
`buildx imagetools inspect img`	Platforms in a tag	18
`docker image prune -a`	Remove unused images	12
`docker build --no-cache --pull .`	Fresh build	16
`build --secret id=x,src=f .`	Build secret	17

VOLUMES, NETWORKS & SYSTEM

COMMAND	DOES	#
`docker volume create|ls|rm|inspect`	Manage volumes	19
`docker network create|ls|rm net`	Manage networks	20
`docker network connect net n`	Attach a container	20
`docker system df [-v]`	Disk usage	28
`docker system prune -a [--volumes]`	Reclaim space	28
`docker builder prune`	Clear build cache	28
`docker context ls · use name`	Switch daemons	03
`docker scout cves img`	Vulnerability scan	29
`docker volume prune [-a]`	Unused volumes	19
`docker network inspect net`	Members, subnet	20
`docker version · info`	Client/server details	02

SWARM, DESKTOP & MODELS

COMMAND	DOES	#
`docker swarm init · join-token worker`	Create / grow cluster	25
`docker node ls · update --availability drain`	Nodes	25
`docker service create|ps|scale|logs`	Services	26
`service update --image i · rollback`	Rolling update	26
`docker stack deploy -c f.yaml name`	Deploy a stack	27
`docker secret create name -`	Secret from stdin	27
`docker desktop start|stop|status`	Control Desktop	04
`docker model pull|run|list`	Local AI models	06
`docker node promote n`	Worker → manager	25
`docker stack ps|services|rm name`	Manage a stack	27
`docker desktop logs|diagnose`	Desktop troubleshooting	04

⚠ COMMON MISTAKES

- Options after the image name go to the app, not Docker
- `prune -a --volumes` deletes unused data volumes
- `localhost` in a container is the container itself

✓ TIPS & BEST PRACTICES

- Most commands accept a unique ID prefix
- Add `--format json` (or a Go template) for scripting
- `docker <cmd> --help` lists every flag

QUICK SUMMARY

- `run` / `exec` / `logs` — daily trio
- `build` / `push` — ship images
- `system df` / `prune` — keep disks clean

Quick Reference — Dockerfile & Compose

Dockerfile instructions · compose.yaml keys · compose CLI · run flag → Compose key · exit codes · Desktop settings paths

DOCKERFILE INSTRUCTIONS

INSTRUCTION	DOES	#
`FROM img AS stage`	Base image / named stage	14
`RUN cmd · RUN --mount=type=cache`	Build step · cached dirs	17
`COPY [--from=s] [--chown=u] src dst`	Add files	14
`WORKDIR /app`	Set working directory	14
`ENV K=V · ARG K=default`	Run+build env · build-only	15
`USER 10001`	Run as non-root	15
`EXPOSE 8080`	Document a port	14
`HEALTHCHECK CMD …`	Health status	15
`ENTRYPOINT ["bin"] · CMD ["args"]`	Program · default args	14
`LABEL k=v · VOLUME /data`	Metadata · mount point	15
`STOPSIGNAL · SHELL`	Stop signal · RUN shell	15
`# syntax=docker/dockerfile:1`	Frontend version	15

COMPOSE CLI

COMMAND	DOES	#
`docker compose up -d [--build]`	Start (rebuild)	21
`docker compose down [-v]`	Remove (+ volumes)	21
`docker compose ps · logs -f svc`	Status · follow logs	21
`docker compose exec svc sh`	Shell in running svc	24
`docker compose run --rm svc cmd`	One-off container	24
`docker compose watch`	Sync / rebuild on change	24
`docker compose -f a -f b config`	Merged final config	24
`docker compose --profile p up`	Enable optional svcs	23
`docker compose up -d --scale w=3`	Run 3 replicas	24
`docker compose build --pull`	Rebuild images	24
`docker compose pull`	Fetch newer images	21
`docker compose ls`	All projects on host	24

COMPOSE.YAML KEYS

KEY	DOES	#
`image · build: { context, args }`	Image or build	21
`ports: ["127.0.0.1:8080:80"]`	Publish ports	22
`environment · env_file`	Container env	22
`depends_on: { db: { condition } }`	Startup order	22
`healthcheck: { test, interval }`	Readiness	22
`volumes · networks · secrets`	Shared resources	23
`profiles: [debug]`	Optional services	23
`develop.watch`	Live sync	24
`deploy: { replicas, resources }`	Limits; Swarm settings	27
`restart: unless-stopped`	Restart policy	22
`command · entrypoint`	Override CMD / ENTRYPOINT	22
`include: [./db/compose.yaml]`	Pull in other files	24

DOCKER RUN FLAG → COMPOSE KEY

FLAG	COMPOSE KEY	#
`-p 8080:80`	`ports`	22
`-v name:/data`	`volumes`	23
`-e K=V · --env-file`	`environment · env_file`	22
`--restart unless-stopped`	`restart`	22
`--network net`	`networks`	23
`--name web`	`container_name`	22
`--memory · --cpus`	`deploy.resources.limits`	22
`--init · --user`	`init · user`	22
`--add-host h:host-gateway`	`extra_hosts`	06
`--read-only · --tmpfs /tmp`	`read_only · tmpfs`	22
`--cap-drop ALL`	`cap_drop`	22
`--label k=v`	`labels`	22

EXIT CODES

CODE	MEANING	#
`0 / 1`	Success / app error	10
`125`	`docker run` itself failed	10
`126 / 127`	Not executable / not found	10
`137`	SIGKILL — often OOMKilled	11
`139 / 143`	Segfault / SIGTERM (stop)	10
`128 + N`	Killed by signal N	10

DOCKER DESKTOP SETTINGS PATHS

PATH	CONTROLS	#
`Resources › Advanced`	CPU, memory, disk	04
`Resources › File sharing`	Bind-mountable folders	04
`General › Use containerd`	Multi-platform images	04
`Docker Engine`	`daemon.json` editor	04
`Kubernetes › Enable`	Local cluster	06
`Resources › Proxies`	HTTP/HTTPS proxy	04

⚠ COMMON MISTAKES

- Inline `#` comments after Dockerfile instructions aren't comments
- `depends_on` without a condition doesn't wait for readiness
- `.env` feeds `${…}`, not containers

✓ TIPS & BEST PRACTICES

- Exec-form `CMD ["…"]` so signals reach the app
- Quote ports and decimals in YAML
- `docker compose config` shows what will run

QUICK SUMMARY

- Dockerfile — build recipe
- `compose.yaml` — multi-container app
- Every run flag has a Compose key